

VCNO

Vibe Coding NO

A Structured Workflow for AI-Assisted Software Development

PROJECT
VCNO

PROJECT VERSION
v1.0

DOCUMENTATION
Official User Guide v1.0.1

LANGUAGE
English · فارسی

REPOSITORY
github.com/MRAVLM/VCNO

LICENSE
MIT

CREATED BY
MRAVLM (Amirali Marjani)

BUILT WITH
AI coding agent assistance

CONTENTS

Table of Contents

Part I — فارسی

فصل ۱ ۱ مقدمه‌ای بر VCNO

فصل ۲ ۲ درک Vibe Coding

فصل ۳ ۳ گردش‌کار VCNO

فصل ۴ ۴ مرحله‌ی /init

فصل ۵ ۵ مرحله‌ی /check

فصل ۶ ۶ مرحله‌ی /PRD

فصل ۷ ۷ مرحله‌ی /Build

فصل ۸ ۸ مرحله‌ی /Feedback

فصل ۹ ۹ مرحله‌ی /Doc

فصل ۱۰ ۱۰ استفاده در یک پروژه‌ی واقعی

فصل ۱۱ ۱۱ کار با ایجنت‌های کدنویس

فصل ۱۲ ۱۲ بهترین شیوه‌ها

فصل ۱۳ ۱۳ اشتباهات رایج

فصل ۱۴ ۱۴ استفاده نکنیم VCNO کِی از

فصل ۱۵ ۱۵ محدودیت‌ها

فصل ۱۶ ۱۶ نسخه‌بندی و نسخه‌های آینده

فصل ۱۷ ۱۷ جمع‌بندی

Part II — English

1 Introduction to VCNO Chapter 1

2 Understanding Vibe Coding Chapter 2

3 The VCNO Workflow Chapter 3

4 The /init Skill Chapter 4

5 The /check Skill Chapter 5

6 The /PRD Skill Chapter 6

7 The /Build Skill Chapter 7

8 The /Feedback Skill Chapter 8

9 The /Doc Skill Chapter 9

10 Using VCNO in a Real Project Chapter 10

11 Working With AI Coding Agents Chapter 11

12 Best Practices Chapter 12

13 Common Mistakes Chapter 13

14 When Not to Use VCNO Chapter 14

15 Limitations Chapter 15

16 Versioning and Future Releases Chapter 16

17 Conclusion Chapter 17

Appendix

A Document Metadata Appendix

فصل ۱

مقدمه‌ای بر VCNO

VCNO چیست؟

VCNO — یا **Vibe Coding NO** — یک گردش‌کار مرحله‌ای و ساخت‌یافته برای توسعه‌ی نرم‌افزار با کمک هوش مصنوعی است. این سیستم، گفت‌وگو با یک ایجنت کدنویس را از یک «چت آزاد» به یک فرایند مهندسی کنترل‌شده تبدیل می‌کند: ابتدا بررسی، سپس اعتبارسنجی، برنامه‌ریزی، پیاده‌سازی، تکرار و در نهایت مستندسازی.

VCNO مجموعه‌ای از *مهارت‌های مبتنی بر پرامپت* است که مشخص می‌کنند ایجنت در هر مرحله چه کاری **باید** و چه کاری **نباید** انجام دهد. به‌جای اینکه به AI بگویید «کل پروژه را بساز»، کار به شش مرحله‌ی صریح شکسته می‌شود.

چرا VCNO به وجود آمد؟

بیشتر توسعه‌های امروزی با AI بدون ساختار هستند: پرامپتی بزرگ فرستاده می‌شود، ایجنت مقدار زیادی کد می‌نویسد، و در نهایت هیچ‌کس مطمئن نیست واقعاً چه چیزی کار می‌کند، چرا تصمیمی گرفته شده، و چه چیزی هنوز ناقص است. VCNO برای حل این مشکل طراحی شده است.

مشکل Vibe Coding کنترل‌نشده

- از دست رفتن زمینه بین گفت‌وگوها
- تولید کد تکراری یا متناقض
- افزودن بی‌رویه‌ی وابستگی‌ها
- گسترش کنترل‌نشده‌ی دامنه (Scope Creep)
- تغییرات مستندنشده
- ادعای موفقیت بدون تأیید واقعی
- ناتوانی در نگهداری پروژه در بلندمدت

فلسفه‌ی VCNO

> اصل بنیادین
>
> قبل از تغییر بفهم. قبل از برنامه‌ریزی اعتبارسنجی کن. قبل از ساخت برنامه‌ریزی کن. قبل از ادعا تأیید کن. واقعیت را مستند کن، نه نیت را.

VCNO با AI مثل یک دستیار مهندسی رفتار می‌کند، نه یک تولیدکننده‌ی کد بی‌کنترل. توسعه‌دهنده کنترل را در دست دارد و AI درون یک فرایند مشخص عمل می‌کند.

VCNO چه چیزی را بهبود می‌دهد؟

- حفظ زمینه بین مراحل
- شفافیت تصمیم‌ها
- قابلیت بازتولید نتایج
- کاهش دوباره‌کاری
- مستندسازی هم‌راستا با واقعیت
- کاهش ریسک تغییرات مخرب

همکاری انسان و AI

در VCNO، AI یک دستیار است — نه یک مرجع بی‌خطا. تصمیم‌های مهم توسط انسان گرفته می‌شوند، و AI در چارچوب مشخصی که خود انسان تعیین کرده عمل می‌کند.

فصل ۲

درک Vibe Coding

Vibe Coding چیست؟

Vibe Coding یعنی ساختن نرم‌افزار با تکیه بر توصیف طبیعی و تکرار سریع، به‌جای نوشتن دقیق همه‌ی جزئیات از پیش. در این سبک، توسعه‌دهنده «حال‌وهوای» محصول را توصیف می‌کند و AI کد را می‌نویسد.

چرا محبوب شد؟

- سرعت بالا در ساخت نمونه‌ی اولیه
- کاهش موانع فنی برای ایده‌پردازی
- امکان تکرار سریع بر اساس بازخورد
- دسترسی به کمک AI در هر مرحله

مزایا

- رسیدن سریع‌تر از ایده به کد کارآمد
- کاهش کارهای تکراری
- یادگیری سریع‌تر برای تازه‌کارها

ریسک‌ها

- **از دست رفتن زمینه:** AI چیزهایی را که قبلاً می‌دانست فراموش می‌کند.
- **توهم (Hallucination):** تولید API یا رفتارهای خیالی.
- **کد تکراری:** پیاده‌سازی دوباره‌ی چیزی که وجود دارد.
- **وابستگی‌های کنترل‌نشده:** افزودن پکیج‌های غیرضروری.
- **گسترش دامنه:** افزودن قابلیت‌های خارج از محدوده.
- **تغییرات مستندنشده:** کدی که کسی دلیلش را نمی‌داند.
- **پیاده‌سازی تأییدنشده:** ادعای موفقیت بدون تست.

VCNO چگونه ساختار می‌دهد؟

VCNO این ریسک‌ها را با تقسیم کار به مراحل صریح و مرزهای سخت کاهش می‌دهد. هر مرحله تنها یک مسئولیت دارد و نمی‌تواند وظایف مرحله‌ی دیگری را تصاحب کند.

فصل ۳

گردش‌کار VCNO

VCNO از شش مرحله تشکیل شده است که به‌ترتیب اجرا می‌شوند. هر مرحله خروجی مرحله‌ی قبل را مصرف می‌کند.



مسئولیت هر مرحله

مرحله	مسئولیت	نباید
`init/`	درک پروژه و تولید سند زمینه	نوشتن کد، نصب، بازآرایی، حذف
`check/`	اعتبارسنجی ایده، نیازمندی‌ها و ریسک‌ها	پیاده‌سازی، طراحی معماری کامل
`PRD/`	تولید نقشه‌ی محصول و فنی	پیاده‌سازی، نصب، مهندسی بیش‌ازحد
`Build/`	پیاده‌سازی تدریجی و تأیید	ساختن نیازمندی جدید، بازنویسی کد کارآمد
`Feedback/`	تکرار کنترل‌شده و رفع باگ	گسترش بی‌صدای دامنه، جایگزینی برنامه‌ریزی
`Doc/`	هم‌راستاسازی مستندات با واقعیت	پیاده‌سازی قابلیت ناقص، پنهان‌کردن شکاف‌ها

> **نکته**
>
> VCNO یک پکیج اجرایی نیست. این سیستم درون یک ایجنت کدنویس سازگار استفاده می‌شود و هر مرحله با فراخوانی نام آن (مثلاً `init/`) شروع می‌شود.

فصل ۴

مرحله‌ی `init/`

هدف

درک پروژه پیش از هرگونه تغییر. این مرحله اولین قدم در VCNO است و باید قبل از هر برنامه‌ریزی یا پیاده‌سازی اجرا شود.

چه زمانی استفاده می‌شود؟

- شروع یک پروژه‌ی جدید از یک ایده
- ورود به یک کدبیس موجود
- راه‌اندازی مجدد VCNO پس از یک تغییر بزرگ
- آماده‌سازی زمینه برای `check/`

چه چیزی را بررسی می‌کند؟

- ساختار مخزن و پوشه‌های مهم
- فایل‌های وابستگی و پیکربندی
- مستندات موجود (/README، docs)
- دارایی‌ها (تصاویر، طراحی، schema، PDF)
- وضعیت Git (شاخه، تغییرات، commits اخیر)
- استک فنی قابل مشاهده

چه چیزی را مستند می‌کند؟

- `docs/project.md` — سند زمینه‌ی پروژه
- `docs/logs/init.md` — گزارش اجرای مرحله
- ساختار مستندات در صورت نبودن

چه کاری نباید انجام دهد؟

> ممنوع
>
> - نوشتن یا تولید کد
> - نصب وابستگی
> - بازآرایی یا بازسازی ساختار
> - حذف فایل‌های کاربر
> - ریست کردن Git یا دور ریختن تغییرات کاربر
> - کپی کردن مقادیر رمز یا توکن در مستندات
> - شروع پیاده‌سازی

نتیجه‌ی مورد انتظار

یک سند زمینه‌ی روشن که تفاوت میان «تأییدشده»، «استنباط‌شده»، «ناشناخته» و «نیاز به توضیح» را حفظ کرده باشد.

نمونه‌ی تعامل

```
کاربر: /init

ایجنت:
  - مخزن را بررسی می‌کند
  - استک را تشخیص می‌دهد
  - مستندات و دارایی‌ها را فهرست می‌کند
  - docs/project.md و docs/logs/init.md را می‌نویسد
  - وضعیت را گزارش می‌دهد: INIT_COMPLETED
  - مرحله‌ی بعد را پیشنهاد می‌دهد: /check
```

فصل ۵

مرحله‌ی `check/`

هدف

اعتبارسنجی ایده، نیازمندی‌ها، محدودیت‌ها، فرض‌ها، امکان‌سنجی، ریسک‌ها و اطلاعات ناقص پیش از شروع `PRD/` .

> پرسش محوری
>
> «آیا آن‌قدر می‌فهمیم چه می‌سازیم که بتوانیم یک PRD قابل‌اعتماد تولید کنیم؟»

ابعاد اعتبارسنجی

- شفافیت و سازگاری نیازمندی‌ها
- قابلیت آزمون‌پذیری
- تناقض‌ها میان نیازمندی‌ها
- گسترش دامنه
- فرض‌های پنهان
- ریسک‌های فنی و محصولی
- ناشناخته‌ها و نیازمندی‌های مبهم

طبقه‌بندی نیازمندی‌ها

دسته	توضیح
Must Have	ضروری برای نسخه‌ی اول
Should Have	مهم ولی قابل تعویق
Could Have	خوب است داشته باشیم
Out of Scope	خارج از دامنه‌ی فعلی
Unknown	نیازمند اطلاعات بیشتر

وضعیت‌های آمادگی

وضعیت	معنا
NOT READY	نیازمندی‌های حیاتی حل نشده‌اند
READY WITH WARNINGS	می‌توان پیش رفت، ولی عدم‌قطعیت‌های غیرمسدودکننده وجود دارد
READY FOR PRD	محصول و جهت فنی به‌قدر کافی روشن هستند

چه کاری نباید انجام دهد؟

> ممنوع
>
> - پیاده‌سازی یا طراحی معماری کامل
> - حل بی‌صدای ابهام مهم
> - تبدیل فرض به واقعیت
> - پرسیدن ده‌ها سؤال بدون اولویت
> - گسترش دامنه به‌جای طبقه‌بندی آن

نتیجه‌ی مورد انتظار

گزارشی شامل نیازمندی‌های تأییدشده، دامنه‌ی MVP، ریسک‌ها، تناقض‌ها، ناشناخته‌ها و وضعیت آمادگی برای `PRD/` .

فصل ۶

مرحله‌ی `PRD/`

هدف

تبدیل فهم معتبر از مراحل پیشین به یک نقشه‌ی آماده‌ی پیاده‌سازی برای `Build/` .

> پرسش محوری
>
> «دقیقاً چه می‌سازیم، چگونه کار می‌کند، و `Build/` چه چیزی را باید پیاده کند؟»

خروجی‌های PRD

- تعریف محصول: هدف، مشکل، کاربران هدف، راه‌حل
- نیازمندی‌های کارکردی و غیرکارکردی
- جریان‌های کاربری
- الزامات UI/UX (در صورت وجود)
- معماری فنی
- استک فناوری
- طراحی پایگاه داده
- طراحی API
- احراز هویت و مجوزدهی
- الزامات امنیتی
- نیازمندی‌های AI/ML (در صورت وجود)
- مدیریت خطا
- حالت‌ها و منطق کسب‌وکار
- ساختار پروژه و وابستگی‌ها
- استراتژی تست
- معیارهای پذیرش
- برنامه‌ی پیاده‌سازی و ترتیب ساخت
- تعریف Done

وضعیت‌های تأیید

وضعیت	معنا
DRAFT	PRD ناقص است
NEEDS CLARIFICATION	تصمیم‌ها یا نیازمندی‌های مهم حل نشده‌اند
READY FOR BUILD	برنامه‌ی پیاده‌سازی به‌قدر کافی روشن است

چه کاری نباید انجام دهد؟

ممنوع

- پیاده‌سازی یا نصب وابستگی
- تغییر بی‌صدای استک فناوری موجود
- مهندسی بیش‌ازحد (microservices، K8s، ... بدون توجیه)
- ادعای موفقیت تست بدون اجرا
- جایگزینی وظایف `check/`

فصل ۷

مرحله‌ی `Build/`

هدف

پیاده‌سازی برنامه‌ی تأییدشده و تبدیل آن به نرم‌افزار کارآمد و تأییدشده.

> پرسش محوری
>
> «چگونه برنامه‌ی تأییدشده را به‌درستی، ایمن و قابل‌تأیید پیاده کنیم؟»

ورودی‌ها

- خروجی `init/`
- خروجی `check/`
- PRD
- وضعیت فعلی مخزن و Git

مراحل اجرا

1. بارگذاری زمینه‌ی پیشین
2. دروازه‌ی آمادگی ساخت
3. بررسی وضعیت Git
4. درک کد موجود پیش از هر تغییر
5. تقسیم کار به فازهای معنادار
6. پیاده‌سازی تدریجی هر فاز
7. تست و تأیید در پایان هر فاز
8. نقاط ذخیره‌ی Git
9. گزارش وضعیت نهایی

وضعیت‌های پایان

وضعیت	معنا
COMPLETE	همه‌ی معیارهای Done برآورده شده‌اند
PARTIAL	بخشی پیاده‌سازی شده ولی همه‌ی معیارها برآورده نشده‌اند
BLOCKED	پیشرفت مسدود شده است

چه کاری نباید انجام دهد؟

> ممنوع
>
> - شروع بدون PRD یا با PRD ناقص
> - بازنویسی معماری کارآمد با معماری دلخواه
> - ساختن نیازمندی جدید
> - افزودن وابستگی به‌صرف راحتی
> - ادعای موفقیت بدون تأیید واقعی
> - جذب مسئولیت‌های `Feedback/`
> - گزارش PARTIAL به‌عنوان COMPLETE

نمونه‌ی چرخه

```
کاربر: /Build

ایجنت:
  - PRD را می‌خواند
  - Git را بررسی می‌کند
  - فازها را اجرا می‌کند
  - در پایان هر فاز تست می‌کند
  - در صورت نیاز checkpoint می‌سازد
  - گزارش نهایی می‌دهد: COMPLETE / PARTIAL / BLOCKED
```

فصل ۸

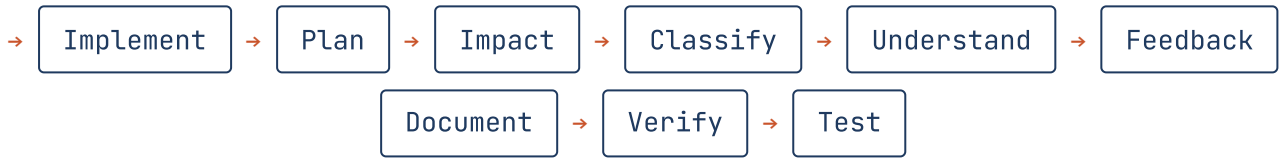
مرحله‌ی `Feedback/`

هدف

تبدیل بازخورد کاربر یا رفتار مشاهده‌شده‌ی پروژه به بهبودهای معتبر، با کنترل دامنه و محافظت در برابر Regression.

> پرسش محوری
>
> «کاربر دقیقاً چه چیزی را می‌خواهد تغییر دهد، چرا، چه چیزی تحت‌تأثیر قرار می‌گیرد، و چگونه بدون شکستن بقیه‌ی پروژه این تغییر را اعمال کنیم؟»

چرخه‌ی تکرار



طبقه‌بندی بازخورد

- Bug
- Feature Request
- UX Improvement
- UI Change
- Performance
- Security
- Data Issue
- Business Logic
- Technical Debt
- Documentation

وضعیت‌های آیتم بازخورد

وضعیت	معنا
NEW / ANALYZING	در حال بررسی
NEEDS CLARIFICATION	نیازمند توضیح بیشتر
PLANNED / IN PROGRESS	در حال برنامه‌ریزی یا اجرا
TESTING / VERIFIED	در حال تست یا تأییدشده
BLOCKED / DEFERRED / REJECTED	مسدود، به‌تعویق‌افتاده، یا ردشده با دلیل

چه زمانی به `check/` برگردیم؟

اگر بازخورد به یک تغییر بزرگ محصولی یا معماری منجر شود (مثلاً «بیایید این را به یک مارکت‌پلیس تبدیل کنیم»)، این مورد باید از طریق `/Build` / `/PRD` → `check` دوباره برنامه‌ریزی شود. `Feedback/` نباید بی‌صدا جای برنامه‌ریزی محصول را بگیرد.

فصل ۹

مرحله‌ی `Doc/`

هدف

هم‌راستاسازی مستندات با پیاده‌سازی واقعی و ارزیابی آمادگی تحویل یا انتشار.

> اصل بنیادین
>
> واقعیت را مستند کن، نه نیت را.

خروجی‌های مستندسازی

- README
- نصب و پیکربندی
- متغیرهای محیطی
- معماری
- ساختار پروژه
- API
- پایگاه داده
- احراز هویت
- استقرار
- تست
- عیب‌یابی
- محدودیت‌های شناخته‌شده
- وضعیت قابلیت‌ها

تفکیک مستندسازی از تأیید

«پیکربندی استقرار وجود دارد» با «استقرار با موفقیت تست شده» یکی نیست. `Doc/` باید همیشه میان «مستندشده»، «تأییدشده»، «تأییدنشده» و «ناشناخته» تفاوت قائل شود.

وضعیت نهایی پروژه

وضعیت	معنا
DOCUMENTED	مستندات با واقعیت هم‌راستا هستند
READY FOR HANDOFF	توسعه‌دهنده‌ی دیگر می‌تواند پروژه را بفهمد و اجرا کند
READY FOR RELEASE	همه‌ی معیارهای انتشار برآورده شده‌اند
PARTIALLY DOCUMENTED	مستندسازی ناقص
BLOCKED	مسدود

چه کاری نباید انجام دهد؟

ممنوع

- پیاده‌سازی قابلیت‌های ناقص
- پنهان‌کردن شکاف‌ها
- ادعای آمادگی انتشار بدون شواهد
- کپی کردن رمزها در مستندات

فصل ۱۰

استفاده در یک پروژه‌ی واقعی

سناریو

یک توسعه‌دهنده می‌خواهد یک اپلیکیشن وب مبتنی بر AI بسازد که به فروشندگان کمک کند پیام‌های مشتریان را تحلیل کنند و دلیل عدم خرید را بفهمند.

مسیر پیشنهادی

1. **ایده:** توسعه‌دهنده ایده را با زبان طبیعی توصیف می‌کند.
2. `init/` : ایجنت مخزن را بررسی می‌کند (احتمالاً خالی است) و زمینه را می‌نویسد.
3. `check/` : نیازمندی‌ها، ابهام‌ها و ریسک‌ها اعتبارسنجی می‌شوند. مثلاً: «تحلیل AI دقیقاً چه چیزی را استخراج می‌کند؟»
4. `PRD/` : نقشه‌ی محصول و فنی تولید می‌شود — مدل داده، API، جریان‌های کاربری، تست.
5. `Build/` : پیاده‌سازی تدریجی: اسکیمای پایگاه داده، API، احراز هویت، رابط کاربری، تست.
6. `Feedback/` : بازخورد کاربر (مثلاً «کارت‌ها خیلی فشرده‌اند») به تغییرات هدفمند تبدیل می‌شود.
7. `Doc/` : مستندات نهایی هم‌راستا با واقعیت پروژه نوشته می‌شوند.

نمونه‌ی تعامل

```
کاربر: می‌خواهم اپلیکیشنی بسازم که به فروشندگان کمک کند
        پیام‌های مشتریان را تحلیل کنند و دلیل عدم خرید را بفهمند.

کاربر: /init
کاربر: /check
کاربر: /PRD
کاربر: /Build
کاربر: /Feedback
کاربر: /Doc
```

نکته

دستورات بالا «نام مراحل» هستند، نه دستورات ترمینال. VCNO یک پکیج CLI نیست؛ این نام‌ها مهارت‌هایی هستند که داخل ایجنت کدنویس فراخوانی می‌شوند.

فصل ۱۱

کار با ایجنت‌های کدنویس

VCNO و ابزارهای ایجنت

VCNO مستقل از ابزار است. مخزن از طریق فایل‌های ورودی بومی هر ابزار به همان منبع واحد اشاره می‌کند:

ابزار	فایل ورودی	اشاره به
Codex, Cursor, Zed, Amp, ...	`AGENTS.md`	منبع اصلی
Claude Code	`CLAUDE.md` + `.claude/skills/*/SKILL.md`	`AGENTS.md` + `/skills`
GitHub Copilot	`github/copilot-instructions.md.`	`AGENTS.md`
Gemini CLI	`GEMINI.md`	`AGENTS.md`
Windsurf	`windsurfrules.`	`AGENTS.md`

مسئولیت انسان

- تعیین هدف و دامنه‌ی پروژه
- پاسخ به سؤالات مبهم
- تأیید PRD پیش از شروع ساخت
- تصمیم‌های مهم معماری و محصولی
- بازبینی نتایج و بازخورد

مسئولیت AI

- بررسی و درک پروژه
- اعتبارسنجی نیازمندی‌ها و طرح سؤال
- تولید PRD
- پیاده‌سازی تدریجی و تست
- پردازش بازخورد
- مستندسازی واقعیت

هشدار

AI را مرجع بی‌خطا ندانید. VCNO جای قضاوت مهندسی انسان را نمی‌گیرد؛ هدف، ساختاریافته‌کردن کمک AI است.

فصل ۱۲

بهترین شیوه‌ها

اصول عملی

- قبل از تغییر بفهم.
- قبل از ساختن بررسی کن.
- قبل از تکرار، دوباره استفاده کن.
- وقتی نیازمندی روشن نیست، بپرس.
- قبل از ادعای موفقیت، تأیید کن.
- دامنه را کنترل کن.
- کار موجود را حفظ کن.
- تغییرات معنادار را تست کن.
- واقعیت را مستند کن.
- از نقاط ذخیره‌ی Git استفاده کن.
- انسان را در تصمیم‌های مهم دخیل نگه دار.

در طول گردش‌کار

- هر مرحله را کامل کنید قبل از رفتن به مرحله‌ی بعد.
- خروجی‌های هر مرحله را به‌عنوان منبع رسمی در نظر بگیرید.
- از بازگشت به مرحله‌ی قبل نترسید اگر اطلاعات جدید ظاهر شد.

در کار با AI

- به AI به‌عنوان دستیار نگاه کنید، نه مرجع.
- از آن بخواهید شواهد تصمیم‌ها را ثبت کند.
- ادعاهای «کار می‌کند» را بدون تأیید نپذیرید.

فصل ۱۳

اشتباهات رایج

- درخواست از AI برای ساختن همه‌چیز به‌صورت یک‌باره.
- رد کردن مرحله‌ی اعتبارسنجی نیازمندی‌ها.
- رد کردن مرحله‌ی برنامه‌ریزی.
- اجازه دادن به گسترش دامنه.
- اعتماد به کد تولیدشده بدون تست.
- بازسازی قابلیت‌هایی که از قبل وجود دارند.
- نادیده‌گرفتن معماری موجود.
- افزودن وابستگی‌های غیرضروری.
- فراموش‌کردن مستندسازی.
- مخلوط‌کردن بازخوردهای نامرتبط.
- تغییرات بزرگ بدون نقاط ذخیره.

> **یادآوری**
>
> هر یک از این اشتباهات در واقع معادل «پریدن از روی یک مرحله از VCNO» است. اگر مرحله‌ای را رد کنید، ضعف‌های آن مرحله دیر یا زود ظاهر می‌شوند.

فصل ۱۴

کِی از VCNO استفاده نکنیم

VCNO برای پروژه‌هایی طراحی شده که به ساختار، زمینه و مستندسازی نیاز دارند. در موارد زیر ممکن است لازم نباشد:

- اسکریپت‌های کوچک یک‌باره
- آزمایش‌های سریع و دورانداختنی
- نمونه‌های اولیه‌ی بسیار ساده
- تغییرات بسیار کوچک روی پروژه‌های موجود

> **نکته**
>
> این بستگی به پیچیدگی پروژه دارد. اگر پروژه در حال رشد است یا باید نگهداری شود، VCNO مفید می‌شود — حتی اگر در ابتدا ساده به‌نظر برسد.

فصل ۱۵

محدودیت‌ها

VCNO صادقانه محدودیت‌های خود را می‌پذیرد:

- تضمین نرم‌افزار بدون باگ نمی‌دهد.
- توهم‌های AI را حذف نمی‌کند.
- جای تست را نمی‌گیرد.
- جای تصمیم‌های انسانی را نمی‌گیرد.
- امنیت را تضمین نمی‌کند.
- نیازمندی‌های مبهم را به‌طور خودکار نمی‌فهمد.
- آمادگی برای تولید را تضمین نمی‌کند.

> **مهم**
>
> VCNO یک چارچوب انسانی است که AI را کنترل می‌کند. موفقیت آن به کیفیت قضاوت انسان و پیگیری منظم بستگی دارد.

فصل ۱۶

نسخه‌بندی و نسخه‌های آینده

نسخه‌ی فعلی

این مستندات **VCNO v1.0** را توصیف می‌کند.

نسخه‌های آینده

نسخه‌های آینده ممکن است مهارت‌های جدید، بهبود گردش‌کار یا تغییرات مستندات را معرفی کنند. هیچ قابلیت مشخصی برای آینده تضمین نمی‌شود.

مخزن رسمی

برای آخرین نسخه و به‌روزرسانی‌ها، مخزن رسمی را دنبال کنید:
https://github.com/MRAVLM/VCNO

> توجه
>
> این مستندات به‌طور خودکار به‌روزرسانی نمی‌شود. برای اطلاع از آخرین وضعیت، به مخزن مراجعه کنید.

فصل ۱۷

جمع‌بندی

هدف VCNO این نیست که AI همه‌چیز را بنویسد.

هدف این است که به AI یک گردش‌کار مهندسی ساخت‌یافته بدهیم و انسان را در کنترل نگه داریم.

> **فلسفه‌ی نهایی**
>
> قبل از تغییر بفهم. قبل از برنامه‌ریزی اعتبارسنجی کن. قبل از ساخت برنامه‌ریزی کن. قبل از ادعا تأیید کن. واقعیت را مستند کن، نه نیت را.

VCNO با AI مثل یک دستیار مهندسی رفتار می‌کند که درون یک فرایند مشخص عمل می‌کند. این سیستم جایگزین قضاوت انسان نمی‌شود؛ ابزاری برای ساختاریافته‌تر کردن کمک AI است.

CHAPTER 1

Introduction to VCNO

What is VCNO?

VCNO — or **Vibe Coding NO** — is a structured, staged workflow for AI-assisted software development. It turns a chat with a coding agent into a controlled engineering process: inspect first, then validate, plan, build, iterate, and document.

VCNO is a collection of *prompt-based skills* that define what an AI coding agent **may** and **may not** do at each stage. Instead of asking an AI to "build this project" in one shot, VCNO breaks the work into six explicit stages.

Why VCNO Exists

Most AI-assisted development today is unstructured: the developer pastes a big prompt, the agent writes a lot of code, and nobody is sure what actually works, why decisions were made, or what is still missing. VCNO was designed to solve this.

The Problem With Uncontrolled Vibe Coding

- Context is lost between conversations
- Duplicate or contradictory code is produced
- Dependencies are added without discipline
- Scope grows uncontrolled (scope creep)
- Changes remain undocumented
- Success is claimed without real verification
- The project becomes unmaintainable over time

VCNO Philosophy

> CORE PRINCIPLE
>
> Understand before changing. Validate before planning. Plan before building. Verify before claiming. Document reality, not intention.

VCNO treats AI as an engineering assistant inside a controlled process — not as an uncontrolled code generator. The developer stays in control.

What VCNO Tries to Improve

- Context preservation across stages
- Transparency of decisions
- Reproducibility of results
- Reduction of rework
- Documentation aligned with reality
- Lower risk of destructive changes

Human + AI Collaboration

In VCNO, AI is an assistant — not an infallible authority. Important decisions are made by the human, and AI operates inside a defined framework set by the human.

CHAPTER 2

Understanding Vibe Coding

What Is Vibe Coding?

Vibe Coding means building software by describing intent in natural language and iterating quickly, rather than writing every detail precisely up front. In this style, the developer describes the "vibe" of the product, and AI writes the code.

Why It Became Popular

- High speed from idea to a first prototype
- Lower technical barriers for ideation
- Fast iteration based on feedback
- AI assistance available at every step

Advantages

- Faster from idea to working code
- Less repetitive work
- Faster learning for beginners

Risks

- **Context loss:** AI forgets what it previously knew.
- **Hallucination:** invented APIs or behaviors.
- **Duplicate code:** re-implementing what already exists.
- **Uncontrolled dependencies:** adding packages for convenience.
- **Scope creep:** adding features outside the intended scope.
- **Undocumented changes:** code nobody can explain later.
- **Unverified implementation:** claiming success without testing.

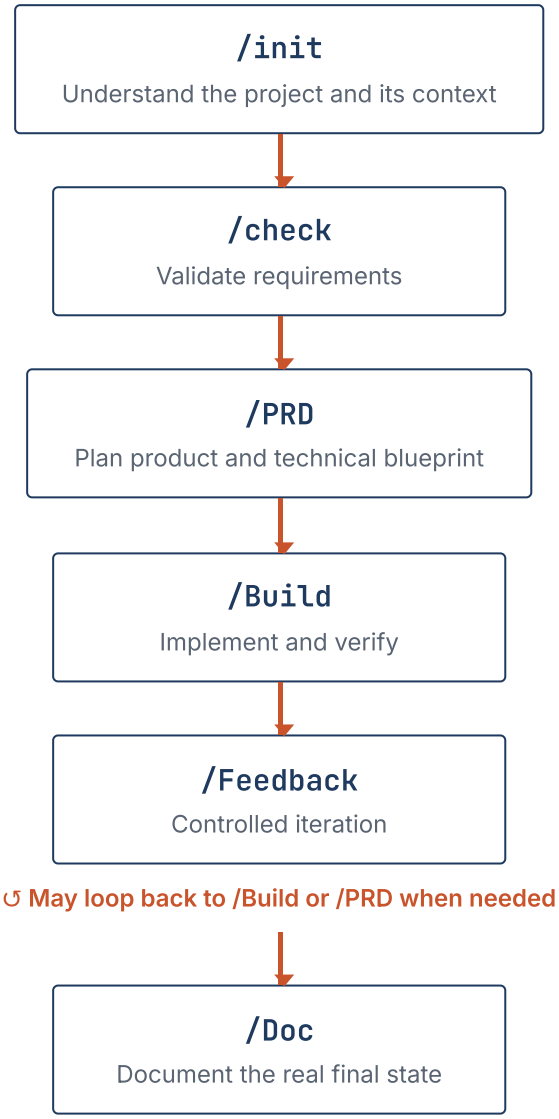
How VCNO Provides Structure

VCNO reduces these risks by breaking work into explicit stages with hard boundaries. Each stage has one responsibility and cannot absorb another's job.

CHAPTER 3

The VCNO Workflow

VCNO consists of six stages executed in order. Each stage consumes the output of the previous one.



Responsibility of Each Stage

Stage	Responsibility	Must not
`/init`	Understand the project and write context	Write code, install, refactor, delete
`/check`	Validate idea, requirements, risks	Implement, design full architecture
`/PRD`	Produce the product/technical blueprint	Implement, install, over-engineer
`/Build`	Implement incrementally and verify	Invent requirements, rewrite working code
`/Feedback`	Controlled iteration and bug fixing	Silently expand scope, replace planning
`/Doc`	Reconcile docs with reality	Implement missing features, hide gaps

> NOTE
>
> VCNO is not an executable package. It is used inside a compatible AI coding agent, and each stage is started by invoking its name (for example `/init`).

CHAPTER 4

The /init Skill

Purpose

Understand the project before any change. This is the first stage of VCNO and must run before any planning or implementation.

When to Use It

- Starting a new project from an idea
- Onboarding an existing codebase into VCNO
- Re-initializing after a major shift
- Preparing context for `/check`

What It Inspects

- Repository structure and important directories
- Dependency and configuration files
- Existing documentation (README, docs/)
- Assets (images, design, PDFs, schemas)
- Git state (branch, tree, recent commits)
- Observable technology stack

What It Documents

- `docs/project.md` — central project context
- `docs/logs/init.md` — stage execution log
- A documentation structure if none exists

What It Must Not Do

FORBIDDEN

- Write or generate code
- Install dependencies
- Refactor or restructure
- Delete user files
- Reset Git or discard user work
- Copy secrets or tokens into documentation
- Start implementation

Expected Result

A clear project context document that keeps Confirmed, Inferred, Unknown, and Needs clarification distinct.

Example Interaction

```
User: /init

Agent:
  - Inspects the repository
  - Detects the stack
  - Lists docs and assets
  - Writes docs/project.md and docs/logs/init.md
  - Reports status: INIT_COMPLETED
  - Suggests next stage: /check
```

CHAPTER 5

The /check Skill

Purpose

Validate the idea, requirements, constraints, assumptions, feasibility, risks, and missing information before `/PRD` begins.

> **CENTRAL QUESTION**
>
> "Do we understand what we are building well enough to produce a reliable PRD?"

Validation Dimensions

- Clarity and consistency of requirements
- Testability
- Contradictions between requirements
- Scope creep
- Hidden assumptions
- Technical and product risks
- Unknowns and ambiguous requirements

Classifying Requirements

Category	Meaning
Must Have	Required for V1
Should Have	Important but deferrable
Could Have	Nice to have
Out of Scope	Outside current scope
Unknown	Needs more information

Readiness States

State	Meaning
NOT READY	Critical requirements are unresolved
READY WITH WARNINGS	Can proceed, but non-blocking uncertainties exist
READY FOR PRD	Product and technical direction are sufficiently clear

What It Must Not Do

FORBIDDEN

- Implement or design the full architecture
- Silently resolve important ambiguity
- Convert assumptions into facts
- Ask dozens of unprioritized questions
- Expand scope instead of classifying it

Expected Result

A report covering validated requirements, MVP scope, risks, contradictions, unknowns, and readiness for `/PRD`.

CHAPTER 6

The /PRD Skill

Purpose

Transform validated understanding from prior stages into an implementation-ready blueprint for `/Build`.

> **CENTRAL QUESTION**
>
> "What exactly are we building, how will it work, and what should `/Build` implement?"

PRD Outputs

- Product definition: purpose, problem, users, solution
- Functional and non-functional requirements
- User flows
- UI/UX requirements (if applicable)
- Technical architecture
- Technology stack
- Database design
- API design
- Authentication and authorization
- Security requirements
- AI/ML requirements (if applicable)
- Error handling
- State and business logic
- Project structure and dependencies
- Testing strategy
- Acceptance criteria
- Implementation plan and build order
- Definition of Done

Approval States

State	Meaning
DRAFT	PRD is incomplete
NEEDS CLARIFICATION	Important decisions or requirements unresolved
READY FOR BUILD	Implementation plan is clear enough

What It Must Not Do

FORBIDDEN

- Implement or install dependencies
- Silently change the existing technology stack
- Over-engineer (microservices, K8s, ... without justification)
- Claim tests passed without running them
- Duplicate the responsibilities of `/check`

CHAPTER 7

The /Build Skill

Purpose

Implement the approved plan and turn it into working, verified software.

> CENTRAL QUESTION
>
> "How do we implement the approved plan correctly, safely, and verifiably?"

Inputs

- Output of `/init`
- Output of `/check`
- The PRD
- Current repository state and Git

Execution Steps

1. Load prior context
2. Build readiness gate
3. Inspect Git state
4. Understand existing code before editing
5. Phase the implementation
6. Implement incrementally
7. Test and verify after each phase
8. Create meaningful Git checkpoints
9. Report final status

Completion States

State	Meaning
COMPLETE	All Definition-of-Done criteria satisfied
PARTIAL	Some work implemented, but not all criteria satisfied
BLOCKED	Progress is blocked

What It Must Not Do

FORBIDDEN

- Start without a PRD, or with an incomplete PRD
- Replace working architecture with preferred architecture
- Invent new requirements
- Add dependencies for convenience
- Claim success without real verification
- Absorb `/Feedback` responsibilities
- Report PARTIAL as COMPLETE

Example Cycle

```
User: /Build

Agent:
  - Reads the PRD
  - Inspects Git
  - Executes phases
  - Tests at the end of each phase
  - Creates checkpoints when appropriate
  - Reports final status: COMPLETE / PARTIAL / BLOCKED
```

CHAPTER 8

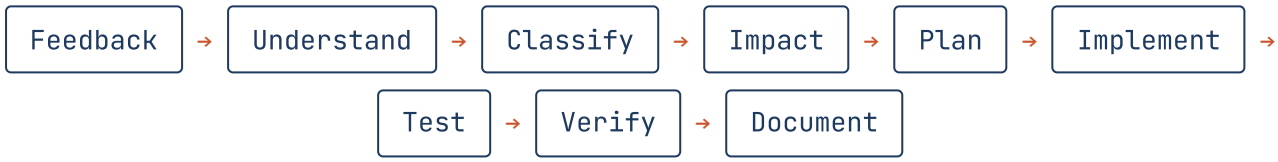
The /Feedback Skill

Purpose

Turn user feedback or observed project behavior into validated improvements, with scope control and regression protection.

> **CENTRAL QUESTION**
>
> "What exactly does the user want changed, why, what does it affect, and how can we change it without breaking the rest of the project?"

The Iteration Loop



Classifying Feedback

- Bug
- Feature Request
- UX Improvement
- UI Change
- Performance
- Security
- Data Issue
- Business Logic
- Technical Debt
- Documentation

Feedback Item States

State	Meaning
NEW / ANALYZING	Under review
NEEDS CLARIFICATION	Requires more information
PLANNED / IN PROGRESS	Being planned or executed
TESTING / VERIFIED	Being tested or verified
BLOCKED / DEFERRED / REJECTED	Blocked, deferred, or rejected with reason

When to Return to `/check`

If feedback leads to a major product or architectural change (for example, "let's turn this into a marketplace"), it must be re-planned through `/check` → `/PRD` → `/Build`. `/Feedback` must not silently replace product planning.

CHAPTER 9

The /Doc Skill

Purpose

Reconcile documentation with the real implementation and evaluate handoff and release readiness.

> CORE PRINCIPLE
>
> Document reality, not intention.

Documentation Outputs

- README
- Installation and configuration
- Environment variables
- Architecture
- Project structure
- API
- Database
- Authentication
- Deployment
- Testing
- Troubleshooting
- Known limitations
- Feature status

Documented vs Verified

"Deployment configuration exists" is not the same as "Deployment was successfully tested." `/Doc` must always distinguish Documented, Verified, Not Verified, and Unknown.

Final Project States

State	Meaning
DOCUMENTED	Documentation matches reality
READY FOR HANDOFF	Another developer can understand and run the project
READY FOR RELEASE	All release criteria are satisfied
PARTIALLY DOCUMENTED	Incomplete documentation
BLOCKED	Blocked

What It Must Not Do

FORBIDDEN

- Implement missing features
- Hide gaps
- Claim release readiness without evidence
- Copy secrets into documentation

CHAPTER 10

Using VCNO in a Real Project

Scenario

A developer wants to build an AI-powered web application that helps shop owners analyze customer messages and understand why people don't buy.

Suggested Path

1. **Idea:** The developer describes the idea in natural language.
2. `/init` **:** The agent inspects the repository (likely empty) and writes context.
3. `/check` **:** Requirements, ambiguities, and risks are validated. For example: "What exactly does the AI analysis extract?"
4. `/PRD` **:** The product and technical blueprint is produced — data model, API, user flows, tests.
5. `/Build` **:** Incremental implementation: DB schema, API, auth, UI, tests.
6. `/Feedback` **:** User feedback (e.g., "the cards feel too crowded") becomes targeted changes.
7. `/Doc` **:** Final documentation is written, aligned with the real project.

Example Interaction

```
User: I want to build an app that helps shop owners
      analyze customer messages and understand why people don't buy.

User: /init
User: /check
User: /PRD
User: /Build
User: /Feedback
User: /Doc
```

> NOTE
>
> The commands above are *stage names*, not terminal commands. VCNO is not a CLI package; these names are skills invoked inside a coding agent.

CHAPTER 11

Working With AI Coding Agents

VCNO and Agent Tools

VCNO is agent-agnostic. The repository points each tool to the same source of truth through its native entry file:

Tool	Entry file	Points to
Codex, Cursor, Zed, Amp, …	`AGENTS.md`	The source
Claude Code	`CLAUDE.md` + `.claude/skills/*/SKILL.md`	`AGENTS.md` + `skills/`
GitHub Copilot	`.github/copilot-instructions.md`	`AGENTS.md`
Gemini CLI	`GEMINI.md`	`AGENTS.md`
Windsurf	`.windsurfrules`	`AGENTS.md`

Human Responsibilities

- Define project goals and scope
- Answer ambiguous questions
- Approve the PRD before building
- Make important architectural and product decisions
- Review results and give feedback

AI Responsibilities

- Inspect and understand the project
- Validate requirements and ask questions
- Produce the PRD
- Implement incrementally and test
- Process feedback
- Document reality

> **WARNING**
>
> Do not treat AI as an infallible authority. VCNO does not replace engineering judgment; it structures AI assistance.

CHAPTER 12

Best Practices

Practical Rules

- Understand before changing.
- Inspect before creating.
- Reuse before duplicating.
- Ask when requirements are unclear.
- Verify before claiming success.
- Keep scope controlled.
- Protect existing work.
- Test meaningful changes.
- Document reality.
- Use Git checkpoints.
- Keep humans involved in important decisions.

Throughout the Workflow

- Complete each stage before moving to the next.
- Treat each stage's output as an official source.
- Do not be afraid to return to an earlier stage if new information appears.

When Working With AI

- Treat AI as an assistant, not an authority.
- Ask it to record the reasoning behind decisions.
- Do not accept "it works" without verification.

CHAPTER 13

Common Mistakes

- Asking AI to build everything at once.
- Skipping requirements validation.
- Skipping planning.
- Allowing scope creep.
- Trusting generated code without testing.
- Rebuilding functionality that already exists.
- Ignoring the existing architecture.
- Adding unnecessary dependencies.
- Forgetting documentation.
- Mixing unrelated feedback.
- Making large changes without checkpoints.

> **REMINDER**
>
> Each of these mistakes is effectively "skipping a stage of VCNO." If you skip a stage, its weaknesses will surface later.

CHAPTER 14

When Not to Use VCNO

VCNO is designed for projects that need structure, context, and documentation. It may be unnecessary for:

- Tiny one-off scripts
- Simple experiments
- Disposable prototypes
- Extremely small changes on existing projects

> **NOTE**
>
> This depends on project complexity. If the project is growing or must be maintained, VCNO becomes useful — even if it looked simple at first.

CHAPTER 15

Limitations

VCNO honestly accepts its limitations:

- It does not guarantee bug-free software.
- It does not eliminate AI hallucinations.
- It does not replace testing.
- It does not replace human decisions.
- It does not guarantee security.
- It does not automatically understand ambiguous requirements.
- It does not guarantee production readiness.

> **IMPORTANT**
>
> VCNO is a human framework that controls AI. Its success depends on the quality of human judgment and consistent follow-through.

CHAPTER 16

Versioning and Future Releases

Current Version

This documentation describes **VCNO v1.0**.

Future Releases

Future releases may introduce new skills, workflow improvements, or documentation changes. No specific future capability is guaranteed.

Official Repository

For the latest version and updates, follow the official repository:
https://github.com/MRAVLM/VCNO

> NOTE
>
> This documentation does not update itself. Check the repository for the current status.

CHAPTER 17

Conclusion

The goal of VCNO is not "let AI write everything."

The goal is to give AI a structured engineering workflow while keeping the human in control.

> **FINAL PHILOSOPHY**
>
> Understand before changing. Validate before planning. Plan before building. Verify before claiming. Document reality, not intention.

VCNO treats AI as an engineering assistant inside a controlled process. It does not replace human judgment; it structures AI assistance.

APPENDIX A

Document Metadata

Document	VCNO Official User Guide
Project	VCNO
Project Version	v1.0
Documentation Version	v1.0.1
Repository	https://github.com/MRAVLM/VCNO
Language	Persian / English
License	MIT
Created by	MRAVLM (Amirali Marjani)
Built with	AI coding agent assistance
Date	2026-10-04 (last updated 2026-10-16)

Verification Status

Item	Status
VCNO v1.0 workflow order	Verified (from README)
Six skills and their responsibilities	Verified (from README and skill files)
Philosophy and principles	Verified (from README)
License (MIT)	Verified (from README)
Author (MRAVLM / Amirali Marjani)	Verified (from README)
Repository URL	Verified
AI tool entry points	Verified (from README)
Exact invocation mechanism in each agent	Needs confirmation (depends on agent)
PDF rendering of this HTML	Verified — generated as vcnobook/VCNO.pdf on 2026-10-16

> **NOTE**
>
> This handbook was written from information available in the VCNO repository as of the date shown. Anything marked "Needs confirmation" or "Not verified" should be checked against the live repository before publishing future revisions.